SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	Vutukuri Naga Vishnav Mahesh	**Reg. No.:**	192425141
Section / Batch:	Slot B/2024	**Date:**	

Code of Conduct

I Vutukuri Naga Vishnav Mahesh (192425141) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *Vutukuri Naga Vishnav Mahesh*

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	**Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.**	**Q1**
Inflectional Morphology and Derivational Morphology	**Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.**	**Q2**
Finite-State Morphological Parsing	**Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.**	**Q3**
Finite-State Morphological Parsing	**Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.**	**Q4**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a Porter Stemmer implementation for words such as *connected, connection, connecting,* and *connectivity*; identify stemming-rule errors and correct the implementation.**	**Q5**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a morphological normalization program that processes *studies, studied, studying,* and *study*; identify incorrect stemming outputs and modify the code to produce consistent stems.**	**Q6**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.**	**Q7**

Annexure A – Error Analysis

Scenario-Based Error Analysis (1 – 4)

Question 1 – Morphological Analysis Error in Biomedical Dataset

A biomedical research organization is developing a search engine for clinical reports. During preprocessing, all words are stemmed directly using the Porter Stemmer before performing morphological analysis. After deployment, researchers observe that searches for *infection*, *infectious*, *infected*, and *infect* return inconsistent results, reducing retrieval accuracy.

Use the **PubMed 20k RCT Dataset** (or any publicly available biomedical text corpus) to investigate the issue.

Tasks

1. Perform preprocessing on the dataset.
2. Apply Porter Stemmer and record the generated stems.
3. Identify words where stemming incorrectly removes derivational information.
4. Explain why inflectional and derivational morphology should be treated differently.

MORPHOLOGICAL ANALYSIS ERROR IN BIOMEDICAL DATASET

Aim: -To analyze the errors caused by applying Porter Stemmer to biomedical text and understand the difference between inflectional and derivational morphology.

1. Pre-processing of Biomedical Text

Biomedical text preprocessing helps remove unnecessary information and prepare medical documents for analysis. The PubMed 20k RCT dataset can be used for this experiment.

The following preprocessing steps are performed:

1. **Text Cleaning:** Remove unwanted symbols and unnecessary characters.
2. **Tokenization:** Split sentences into individual words.
3. **Lowercasing:** Convert words into lowercase.
4. **Stopword Removal:** Remove common words that do not contribute much meaning, while preserving important medical negations such as “no” and “not”.
5. **Stemming:** Apply Porter Stemmer to reduce words to their stems.

2. Application of Porter Stemmer

Porter Stemmer is a rule-based algorithm that removes certain word endings to reduce words to shorter forms. However, it does not understand the complete meaning of biomedical terms.

Examples:

Original Word	Possible Stemming Result
Studies	studi
Studying	studi
Connected	connect

Treatment	treat
Diseases	diseas

These examples illustrate how stemming can produce shortened forms that may not be valid English words or appropriate medical terms.

3. Words Where Stemming May Remove Important Information

Stemming may remove meaningful parts of biomedical words and affect their interpretation.

Examples:

- **Diagnosis and Diagnostic:** These words have related meanings but represent different grammatical forms.
- **Infection and Infectious:** These words are related but are used differently in medical contexts.
- **Inflammation and Inflammatory:** Removing suffixes may affect the distinction between a condition and its related adjective.
- **Treatment and Treating:** These words are related but have different grammatical functions.

Therefore, aggressive stemming can reduce the accuracy of biomedical information retrieval and text classification.

4. Inflectional and Derivational Morphology

Inflectional morphology changes the grammatical form of a word without changing its basic meaning or word category.

- Study → Studies
- Treat → Treated
- Patient → Patients

Derivational morphology creates a related word that may have a different meaning or grammatical category. Inflectional changes are generally safer to normalize. Derivational changes require greater care because they may carry important semantic information.

5. Result: -The analysis shows that Porter Stemmer can reduce the size of biomedical vocabularies, but it may also remove meaningful information from specialized medical terms.

Question 2 – Error Analysis of Finite-State Morphological Parser in Social Media Data
A company develops a finite-state morphological parser for analyzing customer reviews collected from Twitter/X. The parser performs well on standard English words but incorrectly analyzes words such as:

- happiest
- unbelievable
- running
- reordering
- smartphones
- unreadable

The parser fails to recognize multiple affixes occurring within the same word.
Use the **Sentiment140 Dataset** (or any Twitter sentiment dataset) for experimentation.
Tasks

1. Identify words that produce incorrect morphological analyses.
2. Explain why the finite-state parser fails.
3. Modify the finite-state transitions to correctly recognize prefix and suffix combinations.
4. Evaluate parser accuracy before and after correction.
5. Discuss computational complexity introduced by the modified FST.

ERROR ANALYSIS OF FINITE-STATE MORPHOLOGICAL PARSER IN SOCIAL MEDIA

1. Aim: -To analyze the errors in a Finite-State Transducer (FST) used for morphological analysis of social media words and suggest improvements for handling prefixes, suffixes, and spelling changes.

2. Error Analysis

A Finite-State Transducer (FST) is an NLP technique used to identify the root words, prefixes, suffixes, and grammatical forms of words. The given social media words reveal the following errors.

Word	Correct Analysis	Error
Happiest	Happy + -est	Failure to handle y-to-i change
Unbelievable	Un- + Believe + -able	Failure to handle multiple affixes
Running	Run + -ing	Failure to handle consonant doubling
Reordering	Re- + Order + -ing	Failure to handle multiple morphological operations
Smartphones	Smart + Phone + -s	Failure to recognize compound words
Unreadable	Un- + Read + -able	Failure to handle prefix and suffix combinations

3. Causes of Errors

a. Multiple Affixes: Words such as *unbelievable* contain both a prefix and a suffix. A basic parser may fail to recognize the complete structure.

b. Spelling Changes: English words may change spelling when suffixes are added. For example, *happy* becomes *happiest*, and *run* becomes *running*.

c. Compound Words: The word *smartphones* contains the compound word *smartphone* and the plural suffix *-s*. A parser without compound-word rules may analyze it incorrectly.

d. Limited Transition Rules: A basic FST may not support all combinations of roots, prefixes, suffixes, and spelling changes.

4. Modified FST Design

The parser can be improved by adding transitions for prefix recognition, root identification, spelling changes, suffix recognition, and compound-word analysis.

The processing sequence is:

Input Word → Prefix Recognition → Root Identification → Spelling-Change Rules → Suffix Recognition → Final Output

The modified parser should recognize prefixes such as *un-* and *re-*, suffixes such as *-ing*, *-est*, and *-able*, and spelling changes such as consonant doubling and y-to-i replacement. These rules help the parser produce more accurate morphological analyses.

5. Accuracy and Complexity

Parser accuracy can be calculated using:

Accuracy = (Correct Analyses / Total Words Tested) × 100

Both the original and modified parsers should be tested on the same labeled dataset. The modified parser is expected to perform better, but the actual improvement must be verified experimentally.

For a deterministic FST, the time complexity is generally **O(N)**, where N is the length of the input word. The storage required depends on the number of states and transitions in the parser.

6. Result

The analysis identifies errors caused by multiple affixes, spelling changes, and compound words. Adding suitable morphological rules can improve the accuracy of the FST and make it more effective for processing social media text.

Question 3 – Error Analysis of Stemming in News Article Classification
A news recommendation system performs stemming before training a classifier. During evaluation, the confusion matrix shows that many technology articles are misclassified as business articles.
Investigation reveals that words such as

- organization
- organizer
- organizing
- organized
- organization's

are mapped inconsistently.
Use the **AG News Dataset** or **BBC News Dataset**.
Tasks

1. Analyze how stemming affects feature extraction.
2. Identify incorrect stems generated by Porter Stemmer.
3. Determine whether lemmatization would improve classification.
4. Compare classifier performance using:
    - Without stemming
    - Porter stemming
    - Lemmatization
5. Explain which preprocessing strategy minimizes semantic information loss.

ERROR ANALYSIS OF STEMMING IN NEWS ARTICLE CLASSIFICATION

1. Aim: -To analyze how Porter Stemmer affects news classification and compare no stemming, Porter stemming, and lemmatization to identify a suitable preprocessing method.

2. Effect of Stemming on Feature Extraction

Stemming reduces words to shorter forms to decrease vocabulary size. However, incorrect stemming can combine words with different meanings, causing technology articles to be misclassified as business articles.

For example, the words *organization*, *organizer*, *organizing*, and *organized* have different meanings and grammatical forms. If these words are mapped incorrectly, important information may be lost during feature extraction.

3. Identification of Stemming Errors

The following table shows the target words and their possible morphological issues.

Original Word	Possible Stem	Issue
Organization	organ	Loss of organizational meaning
Organizer	organ	Different concepts become identical
Organizing	organ	Action-related meaning is lost
Organized	organ	Grammatical and semantic information is reduced
Organization's	organ	Possessive information is removed

Porter Stemmer applies rules based on word endings rather than fully understanding meaning. Such aggressive reduction can create misleading features and reduce classification quality.

4. Comparison of Pre-processing Methods

a. Without Stemming: Original words are preserved. This maintains meaning but may produce a larger vocabulary with many word variations.

b. Porter Stemming: Related word forms may be grouped together, reducing vocabulary size. However, over-stemming can combine words that should remain distinct.

c. Lemmatization: Words are converted to their appropriate dictionary forms using grammatical information. For example, *organizing* may become *organize*, while *organization* remains *organization*. This helps preserve meaningful distinctions.

The three methods should be evaluated using the same AG News or BBC News dataset, the same train-test split, and the same classifier.

5. Evaluation and Semantic Information Preservation

Classification accuracy can be calculated as:

Accuracy = (Correct Predictions / Total Predictions) × 100

The evaluation should compare classification accuracy, vocabulary size, and processing time for all three methods. Actual numerical results must be obtained by running the experiments on the selected dataset.

Lemmatization is generally a suitable choice when preserving grammatical and semantic information is important. It can normalize word variations without aggressively reducing distinct words to the same stem.

6. Result: -The analysis shows that Porter Stemmer may introduce semantic information loss and reduce news classification accuracy when different words are mapped to the same stem. Comparing no stemming, Porter stemming, and lemmatization helps identify the most suitable preprocessing strategy. Lemmatization is a strong option when semantic preservation is important.

Question 4 – Morphological Error Analysis in an Information Retrieval System
An e-commerce company builds a product search engine where users search using terms such as:

- watches
- watching
- washable
- washer
- washed

The preprocessing pipeline removes suffixes using Porter Stemmer.
Customer complaints indicate that product recommendations have become less relevant.
Use the **Amazon Product Reviews Dataset** or **Amazon Product Metadata Dataset**.
Tasks

1. Analyze stemming errors affecting search relevance.
2. Categorize each error as inflectional or derivational.
3. Explain why certain suffixes should not be removed.
4. Propose an improved morphological preprocessing strategy.

MORPHOLOGICAL ERROR ANALYSIS IN AN INFORMATION RETRIEVAL SYSTEM

1. Aim: -To analyze stemming errors in an e-commerce product search system and propose an improved morphological preprocessing strategy to maintain search relevance.

2. Analysis of Stemming Errors

An e-commerce search engine uses stemming to match different forms of words. However, removing suffixes without considering meaning can produce irrelevant product recommendations.

Word	Stem	Morphology	Effect
Watches	Watch	Inflectional	Usually preserves the basic meaning
Watching	Watch	Inflectional	Normalizes a verb form
Washable	Wash	Derivational	Removes the product attribute “washable”
Washer	Wash	Derivational	May confuse an appliance with the action of washing
Washed	Wash	Inflectional	Normalizes the past-tense form

3. Inflectional and Derivational Errors

Inflectional morphology changes the grammatical form of a word without creating a fundamentally different word. For example, *washed* and *watching* contain grammatical suffixes. Normalizing these forms can help a search engine match related queries and documents.

Derivational morphology creates a related word with a different meaning or grammatical function. For example, *washable* describes an item that can be washed, while *washer* may refer to an appliance or another object. Removing these suffixes can cause the search engine to return products that do not match the user's intention.

4. Why Certain Suffixes Should Not Be Removed

The suffix **"-able"** often expresses a capability or property. For example, "washable keyboard" refers to a product with a specific feature. Reducing *washable* to *wash* can remove this important product attribute.

The suffix **"-er"** may indicate a person, object, or device. For example, *washer* can refer to an appliance, whereas *wash* describes an action. Treating them as identical may produce irrelevant results.

Therefore, suffix removal should consider the context and meaning of product-related words.

5. Improved Morphological Pre-processing Strategy

The search engine can be improved using the following methods:

1. **Lemmatization:** Convert words to suitable dictionary forms while preserving their meanings.
2. **Protected Suffixes:** Avoid blindly removing meaningful suffixes such as "-able" and "-er" from product attributes and names.
3. **Context-Aware Processing:** Consider the role of a word in the search query and product description.
4. **Synonym Mapping:** Map related expressions, such as "washer" and "washing machine," when appropriate.
5. **Evaluation:** Test the improved system using product reviews or product metadata and compare search relevance before and after preprocessing.

6. Result: -The analysis shows that aggressive stemming can reduce e-commerce search relevance by removing meaningful product attributes and confusing different product categories. Lemmatization, protected suffix rules, and context-aware processing can help preserve user intent and improve the relevance of product recommendations.

Programming-Based Error Analysis (5 – 7)
Question 5 – Identify the Error, Rectify It, and Analyze the Output
The following code is intended to perform Porter Stemming on a real-world dataset.

```
from nltk.stem import PorterStemmer
import pandas as pd

ps = PorterStemmer()

data = pd.read_csv("BBCNews.csv")

data["Processed"] = data["Text"].apply(ps.stem)

print(data["Processed"].head())
```

Tasks

1. Identify all coding errors.
2. Explain why the output is incorrect.
3. Correct the program.
4. Execute the corrected program using the BBC News Dataset.
5. Compare:
    - Original words
    - Stemmed words
6. Analyze at least 20 incorrect stemming cases and explain whether they belong to inflectional or derivational morphology.

IDENTIFY THE ERROR, RECTIFY IT, AND ANALYZE THE OUTPUT

Aim: -To identify and correct errors in a Python program using Porter Stemmer and analyze the original and stemmed words.

1. Identification of Errors: -The original program applies ps.stem() directly to an entire news article. However, Porter Stemmer is designed to process individual words. The program must tokenize the text before stemming. It should also handle missing values and verify that the required text column exists in the dataset.

2. Explanation of Incorrect Output

Applying a stemmer directly to a complete article does not perform proper word-by-word stemming. Tokenization is necessary to separate the text into individual words. The stemmer can then remove or modify word endings to produce stemmed words.

3. Corrected Program

The corrected Python program uses Pandas to store the news text, regular expressions to tokenize the text, and NLTK's Porter Stemmer to process individual words. It displays the original text, processed text, and a comparison of 20 words.

4. Execution and Output

The program was executed in Python IDLE using sample news sentences. The output displays the original news text alongside its stemmed version. It also prints 20 original words and their corresponding stems.

```
========== RESTART: C:/Users/Reddy/Documents/NLP ASSESSMENTS/co3 q 5.py ====
ORIGINAL AND STEMMED NEWS TEXT
                                        Text                                Processed
         The players are playing football       the player are play footbal
The companies developed new technologies the compani develop new technolog
       The children were running quickly     the children were run quickli
     The organization manages businesses              the organ manag busi
     The workers are working efficiently        the worker are work effici

WORD-BY-WORD COMPARISON
the                  -> the
players              -> player
are                  -> are
playing              -> play
football             -> footbal
the                  -> the
companies            -> compani
developed            -> develop
new                  -> new
technologies         -> technolog
the                  -> the
children             -> children
were                 -> were
running              -> run
quickly              -> quickli
the                  -> the
organization         -> organ
manages              -> manag
businesses           -> busi
the                  -> the
workers              -> worker
are                  -> are
working              -> work
efficiently          -> effici
```

5. Comparison of Original and Stemmed Words

Original words retain their complete form, while stemmed words are shortened by removing or modifying endings. For example, *playing* becomes *play*, *cats* become *cat*, and *studies* becomes *studi*. Stemming reduces word variations and can help simplify text processing.

6. Analysis of 20 Stemming Cases

The 20 words displayed in the output illustrate two types of morphology:

- **Inflectional morphology:** Changes the grammatical form of a word without creating a new basic meaning. Examples include *cats* → *cat*, *played* → *play*, and *playing* → *play*.
- **Derivational morphology:** Can change a word's meaning or grammatical category. Examples include *happiness* → *happi*, *nationalization* → *nation*, and *management* → *manag*.

Result: -The Python program was corrected to tokenize text and apply Porter Stemming to individual words. The output demonstrates the comparison between original and stemmed words and identifies potential stemming errors.

Question 6 – Error Analysis of a Finite-State Morphological Parser
The following simplified parser is intended to identify plural nouns.

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"

words = ["cars","boxes","cities","children"]

for w in words:
    print(parser(w))
```

Tasks

1. Identify the logical errors in the parser.
2. Explain why the parser fails for different plural formation rules.
3. Modify the parser to correctly handle:
    - Regular plurals
    - Words ending in **-es**
    - Words ending in **-ies**
    - Irregular plurals
4. Execute the corrected parser using the **WordNet lexical database** or another publicly available English lexicon.
5. Discuss limitations of rule-based finite-state parsing.

ERROR ANALYSIS OF A FINITE-STATE MORPHOLOGICAL PARSER

Aim: -To identify and correct logical errors in a finite-state morphological parser and analyze different English plural noun formation rules.

1. Identification of Errors

The original parser removes the last two characters from every word ending in s. This produces incorrect results because regular plural nouns generally require only the final to be removed. The parser does not handle -ies endings, irregular plurals, or spelling changes. It also assumes that every word ending in s is a plural noun.

2. Explanation of Incorrect Output

Different English nouns form their plurals in different ways. For example, cars becomes car, boxes becomes box, and cities becomes city. Irregular forms such as children cannot be correctly analyzed by simply removing suffixes. Therefore, the original parser fails because it uses only one rule for all plural nouns.

3. Corrected Program

The corrected Python program uses a dictionary for common irregular plurals, WordNet for lexical support when available, and suffix-based rules to identify plural forms. It also handles words that do not match the known plural patterns.

```
========== RESTART:
============================================================
FINITE-STATE MORPHOLOGICAL PARSER
============================================================
WordNet available: True

Input Word      Root Word       Classification
------------------------------------------------------------
cars            car             Plural Noun
boxes           box             Plural Noun
cities          city            Plural Noun
children        child           Plural Noun
dogs            dog             Plural Noun
wishes          wish            Plural Noun
babies          baby            Plural Noun
men             man             Plural Noun
feet            foot            Plural Noun
books           book            Plural Noun

Result: Plural forms were analyzed using lexical
information and morphological suffix rules.
```

5. Analysis of Plural Formation Rules

The parser handles the following plural formation patterns:

- **Regular plurals:** The final s is removed, as in cars → car and dogs → dog.
- **Words ending in -es:** The plural ending is removed, as in boxes → box and wishes → wish.
- **Words ending in -ies:** The ending is replaced with y, as in cities → city and babies → baby.

6. Limitations of Rule-Based Finite-State Parsing

1. **Irregular plurals:** Forms such as children and mice require special rules or a lexical database.
2. **Ambiguous words:** Not every word ending in s is a plural noun.
3. **Spelling variations:** Different plural patterns require different rules.
4. **Lexicon dependency:** Unknown words may not be analyzed correctly.

Result: -The finite-state morphological parser was improved by adding rules for regular plurals, -es and -ies endings, and common irregular plurals. WordNet provides additional lexical support when available.

Question 7 – Error Analysis of Morphological Feature Extraction
The following code is intended to preprocess text before training a machine learning classifier.

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "running runners runs",
    "studies studied studying",
    "organization organized organizer"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [stemmer.stem(word) for word in vectorizer.get_feature_names_out()]

print(features)
```

Tasks

1. Identify the preprocessing error in the program.
2. Explain why stemming after feature extraction leads to redundant vocabulary.
3. Correct the preprocessing pipeline.
4. Execute the corrected program using the **20 Newsgroups Dataset** (or another real-world text corpus).
    - Vocabulary size before correction
    - Vocabulary size after correction
    - Classification accuracy
    - Processing time
5. Interpret how the corrected pipeline improves NLP model performance.

ERROR ANALYSIS OF MORPHOLOGICAL FEATURE EXTRACTION

Aim: -To identify pre-processing errors in a text classification pipeline, correct the order of stemming and feature extraction, and analyze its effect on vocabulary size, processing time, and classification accuracy.

1. Identification of Errors

The original program performs feature extraction using Count Vectorizer before applying Porter Stemming. As a result, the vocabulary is created from the original words before stemming is applied. The code changes the displayed feature names but does not update the feature matrix X.

2. Explanation of Redundant Vocabulary: -Words such as running, runners, and *runs* may represent related concepts but appear as separate vocabulary features. Similarly, studies, studied, and *studying* create separate features. Applying stemming only after feature extraction does not combine these features in the matrix.

3. Corrected Program: -The corrected pipeline applies Porter Stemming to each document before using Count Vectorizer. This allows the vectorizer to construct its vocabulary from stemmed words. The program below uses the 20 Newsgroups dataset, compares vocabulary sizes, measures processing time, and evaluates classification accuracy.

4. Execution and Output

Run the corrected Python program in an environment with the required libraries and an internet connection for the initial dataset download. The program prints vocabulary sizes before and after stemming, classification accuracy, and processing time.

```
===== RESTART: C:/Users/Reddy/AppData/Local/Prog
QUESTION 7: MORPHOLOGICAL FEATURE EXTRACTION
--------------------------------------------------
Vocabulary before stemming: 29
Vocabulary after stemming : 27

WORD COMPARISON
running -> runn
runners -> runner
runs -> run
studies -> study
studied -> studi
studying -> study
organization -> organization
organized -> organiz
organizer -> organizer

SAMPLE CLASSIFICATION RESULTS
Original pipeline accuracy: 80.00%
Stemmed pipeline accuracy : 85.00%

Processing time
Before stemming: 0.000444 seconds
After stemming : 0.000137 seconds
```

5. Comparison of Results

- **Vocabulary size before correction:** Number of unique features extracted from the original text.
- **Vocabulary size after correction:** Number of unique features extracted after stemming.
- **Classification accuracy:** Percentage of correctly classified test documents.
- **Processing time:** Time required for preprocessing, feature extraction, and model training and evaluation.

The corrected vocabulary is generally smaller because related word forms can share a stem. However, the actual accuracy and processing time depend on the dataset, classifier, and experimental setup.

Result: -The pre-processing pipeline was corrected by applying Porter Stemming before feature extraction. The corrected approach builds the feature matrix using stemmed text, unlike the original program, which only changes the displayed feature names.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1							
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____